

GreenLight OA — Study Guide (Java + SQL)

Organized by **concept**, not question number — the OA is testing about 10 core traps in Java and 10 in SQL, repeated across 40 questions. Learn the trap once, and you'll get every question built on it.

PART 1: JAVA — Grouped by Concept

Concept A: Integer/String caching & `==` vs `.equals()` (Q1, Q3)

The rule: `==` compares references (memory addresses) for objects; `.equals()` compares content.

- **Integer caching (Q1):** Java caches `Integer` objects for values **-128 to 127**. So `Integer a=127, b=127` → same cached object → `a==b` is `true`. But `200` is outside the cache range → two separate objects → `c==d` is `false`. **Answer: B (true, false).**
- **String interning (Q3):** String literals (`"hello"`) live in a shared "string pool" — two literals with the same text point to the same object, so `s1==s2` is `true`. But `new String("hello")` forces a **new heap object**, so `s1==s3` is `false`. `.equals()` always compares content → `true`. **Answer: B (true, false, true).**

Brush up: memorize the -128 to 127 Integer cache range. This trap appears in almost every Java OA.

Concept B: Static vs instance method resolution / polymorphism (Q2, Q21)

The rule: Instance methods use **runtime polymorphism** (resolved by actual object type). Static methods use **compile-time resolution** (resolved by the reference/declared type) — they are never truly "overridden," only **hidden**.

- **Q2:** `Base b = new Derived();` `b.show()` is static → resolved by `Base` (declared type) → prints "Base static". `b.display()` is instance → resolved by actual object (`Derived`) → prints "Derived display". **Answer: B.**
- **Q21 (harder — constructor + polymorphism trap):** `A`'s constructor calls `init()`, which is overridden in `B`. Since `init()` is virtual, `B.init()` runs — but this happens **before** `B`'s field initializers (`x=10`) execute, because in Java, initialization order is: (1) superclass constructor fully runs first, (2) *then* subclass field initializers run, (3) *then* subclass constructor body runs. So when `A()` calls `init()`, `x` is still its default value `0`. **Answer: B ("A" / "B init, x=0" / "B, x=10").**

Brush up: the golden rule — **never call overridable methods from a constructor**. This exact gotcha (static hiding vs dynamic dispatch, and constructor-calls-overridden-method) is a classic OA/interview favorite.

Concept C: `try`/`catch`/`finally` semantics (Q5, Q12, Q15)

The rule: `finally` **always** runs — even if there's no matching catch, even on `return`/`continue`/`break`, even if an exception propagates uncaught.

- **Q5:** No catch block for `RuntimeException`. `finally` still runs and prints before the exception propagates further up (crashing the program after). **Answer: A.**
- **Q12 (compile-time trap):** Catching `Exception` (general) **before** `ArrayIndexOutOfBoundsException` (specific, and a subclass of `Exception`) makes the second catch **unreachable code** — a compile-time error, not a runtime one. **Answer: C.**
- **Q15 (finally + continue trace):** Walk it index by index:
 - `i=0`: no exception, `result="0"`, finally adds "F" → `"0F"`
 - `i=1`: exception thrown → catch adds "E" → `"0FE"`, finally still runs → `"0FEF"`, then `continue`
 - `i=2`: `result+="2"` → `"0FEF2"`, finally adds "F" → `"0FEF2F"`
 - **Answer: A (0FEF2F)**

Brush up: internalize "finally ALWAYS executes" as an absolute rule, and "more specific catch types must come before more general ones" as a compile-time rule.

Concept D: Operator evaluation order (pre/post increment) (Q7, Q13)

The rule: `x++` returns the *current* value then increments; `++x` increments *first* then returns. Java evaluates left-to-right.

- **Q7:** `x=5`. `x++` → returns 5, x becomes 6. `++x` → x becomes 7, returns 7. `y = 5+7 = 12`. Final `x=7`. **Answer: B (7 12).**
- **Q13:** Three calls to `counter++` (post-increment) in left-to-right order return `0, 1, 2` → sum = 3. After all three calls, `counter=3`. **Answer: A (3 3).**

Brush up: Practice tracing 3-4 of these by hand — draw a table with columns (expression, value used, value after). This is a very common "gotcha" style question.

Concept E: Collections — mutation during iteration, thread safety, ordering (Q4, Q9, Q14)

- **Q4 (thread-safe map choice):** For high-read/low-write concurrent access, `ConcurrentHashMap` uses fine-grained locking (segment/bucket-level or CAS), far better throughput than `Hashtable` or `Collections.synchronizedMap()`, both of which lock the *entire* map on every access. **Answer: C.**
- **Q9 (ConcurrentModificationException):** Removing from a `List` during a for-each loop invalidates the iterator's internal `modCount` tracking → throws `ConcurrentModificationException`. Fix: use `Iterator.remove()` or `list.removeIf(...)`. **Answer: A (throws CME).**

- **Q14 (Map ordering/complexity):** `HashMap` = no order guarantee, $O(1)$ avg. `LinkedHashMap` = insertion order, $O(1)$ avg. `TreeMap` = sorted by key (natural/Comparator), $O(\log n)$ — backed by a red-black tree. **Answer: B.**

Brush up: Know this table cold: `HashMap` (no order, $O(1)$) vs `LinkedHashMap` (insertion order, $O(1)$) vs `TreeMap` (sorted, $O(\log n)$). Also know `ConcurrentHashMap` vs `Hashtable` vs `synchronizedMap`.

Concept F: Mutable reference semantics — `StringBuilder` (Q11)

The rule: `StringBuilder.append()` **mutates the object in place and returns the same reference.** Reassigning the original variable name to a *new* object afterward does not affect other variables that still hold the *old* reference.

- **Q11:** `sb2 = sb1.append("def")` → `sb2` and `sb1` point to the *same* mutated object ("abcdef"). Reassigning `sb1 = new StringBuilder("xyz")` only changes what `sb1` points to — `sb2` is untouched. **Answer: B (abcdef).**

Brush up: this is really about understanding **references vs values** — same idea trips people up with arrays and mutable objects generally, not just `StringBuilder`.

Concept G: Autoboxing / numeric promotion trap (Q17)

The rule: In a ternary `cond ? A : B`, if the two branch types are different **boxed numeric types** (e.g., `Integer` and `Double`), Java unboxes *both* and promotes to the wider type (`double`) as part of determining the ternary's static type — **regardless of which branch is actually chosen at runtime.**

- **Q17:** `true ? new Integer(1) : new Double(2.0)` → both operands get unboxed/promoted to `double` at compile time → result is `1.0`, not `1`. **Answer: B (1.0).**

Brush up: this is obscure but a known "gotcha" category — mixed numeric types in ternaries. Just remember the rule; don't try to derive it under pressure.

Concept H: Exception hierarchy (Q18)

The rule: `Throwable` → `Exception` and `Error`. `Exception` → `RuntimeException` (unchecked) and other subclasses like `IOException` (checked, must be declared/caught).

- **Q18: Answer: C** — `RuntimeException` extends `Exception`; `RuntimeException` subclasses are unchecked, other `Exception` subclasses are checked.

Brush up: draw this hierarchy tree from memory: `Throwable` → {`Error`, `Exception` → {`RuntimeException` (unchecked), `IOException`/`SQLException`/etc (checked)}}.

Concept I: `BigDecimal` for currency (Q6) — fintech-specific

- **Q6:** `double` uses binary floating-point, which cannot exactly represent many decimal fractions (e.g., 0.1) → rounding errors, unacceptable for money. `BigDecimal` (with explicit scale + `RoundingMode`) is standard for currency. **Answer: C.**

Brush up: Given this is a fintech OA, expect follow-up depth here — know

`BigDecimal.setScale()`, `RoundingMode.HALF_UP`, and why you should construct `BigDecimal` from a `String` (`(new BigDecimal("0.1"))`) not a `double` (`(new BigDecimal(0.1))` inherits the binary imprecision).

Concept J: Singleton thread-safety (Q8)

- **Q8:** Lazy singleton with no synchronization → two threads can both pass `if(instance==null)` simultaneously → two separate instances created (race condition). **Answer: B.** Fixes: double-checked locking + `volatile`, enum singleton, or static holder class (initialization-on-demand holder idiom).

Brush up: be ready to write the double-checked locking fix if asked to code it:

```
java

private static volatile ConfigManager instance;
public static ConfigManager getInstance() {
    if (instance == null) {
        synchronized (ConfigManager.class) {
            if (instance == null) instance = new ConfigManager();
        }
    }
    return instance;
}
```

Concept K: Functional interfaces (Q10)

- **Q10:** A lambda taking one argument and returning `boolean` = `Predicate<T>`. **Answer: C.**

Brush up: know the four core ones: `Function<T,R>` (takes T, returns R), `Predicate<T>` (takes T, returns boolean), `Consumer<T>` (takes T, returns void), `Supplier<T>` (takes nothing, returns T).

Concept L: `volatile` semantics (Q20)

- **Q20:** `volatile` guarantees **visibility** (all threads see the latest write) but **not atomicity** — `i++` is still a read-modify-write sequence that can race. **Answer: B.**
-

Concept M: Stream laziness (Q19)

- **Q19:** Intermediate stream operations (`filter`, `map`) are **lazy** — they don't execute until a terminal operation (`count()`, `collect()`, etc.) is called. So `"Stream created..."` prints *before* any filter logic runs. **Answer: B.**

Brush up: know the intermediate (lazy: `filter`, `map`, `sorted`) vs terminal (`collect`, `count`, `forEach`, `reduce`) distinction.

PART 2: SQL — Grouped by Concept

Concept A: WHERE vs HAVING (Q1, Q6)

The rule: `WHERE` filters rows **before** grouping/aggregation — you cannot use aggregate functions (`SUM`, `COUNT`, etc.) in `WHERE`. `HAVING` filters **after** grouping, and *can* reference aggregates.

- **Q1: Answer: B.**
 - **Q6 (harder — balance reconciliation):** `HAVING a.balance <> COALESCE(SUM(t.amount), 0)` finds accounts where the stored balance doesn't match the summed transaction ledger — a real fintech reconciliation pattern. Since `a.balance` is in the `GROUP BY` list, referencing it in `HAVING` is valid. **Answer: A.**
-

Concept B: JOIN types (Q2)

- **Q2:** `LEFT JOIN` keeps **all** rows from the left table, filling unmatched right-side columns with `NULL`. `INNER JOIN` only returns matched rows. **Answer: B.**

Brush up: know INNER, LEFT, RIGHT, FULL OUTER, and what NULLs appear where in each.

Concept C: Window functions — RANK vs DENSE_RANK vs ROW_NUMBER (Q3, Q13, Q14)

The rule (memorize this table for ties, e.g. values [100,100,90,80]):

Function	Ties behavior	Result for [100,100,90,80]
<code>ROW_NUMBER()</code>	always unique, sequential	1,2,3,4
<code>RANK()</code>	same rank for ties, skips next rank(s)	1,1,3,4
<code>DENSE_RANK()</code>	same rank for ties, no skip	1,1,2,3

- **Q3 (2nd highest per account, portable):** `LIMIT/OFFSET` isn't standard SQL and doesn't partition per group. `DENSE_RANK() OVER (PARTITION BY account_id ORDER BY amount DESC)` filtered to rank=2 is the correct portable approach. **Answer: C.**
- **Q13: Answer: C (ROW_NUMBER).**
- **Q14: Answer: C (1,1,3,4).**

Brush up: this table is probably the single highest-value thing to have memorized cold before the OA — it appears 3 times here alone.

Concept D: NULL handling / three-valued logic (Q9, Q10, Q12)

The rule: SQL has three truth values: TRUE, FALSE, UNKNOWN. Any comparison with `NULL` (like `= NULL`) evaluates to UNKNOWN, never TRUE.

- **Q9 (COUNT variants):** `COUNT(*)` counts all rows (100). `COUNT(column)` only counts **non-null** values in that column. If `closed_date` is NULL for 80 of 100 rows, `COUNT(closed_date) = 20`. **Answer: C (100, 100, 20).**
- **Q10:** `WHERE amount = NULL` always returns UNKNOWN → zero rows, always. Must use `IS NULL`. **Answer: B.**
- **Q12 (NOT IN + NULL gotcha — very important, very common trap):** If the subquery feeding `NOT IN` returns **even one NULL** in its result set, the *entire* `NOT IN` comparison becomes UNKNOWN for every outer row → query silently returns **zero rows**. This is a classic silent-bug pattern. `NOT EXISTS` doesn't have this problem. **Answer: B.**

Brush up: Rule of thumb to memorize: **prefer** `NOT EXISTS` **over** `NOT IN` whenever the subquery column could contain NULLs — this is one of the most common real-world SQL bugs, so OAs love testing it.

Concept E: Correlated subqueries — EXISTS/NOT EXISTS (Q11)

- **Q11:** To find customers who **never** transacted, use `NOT EXISTS` with a correlated subquery joining `Transactions` → `Accounts` → `customer_id`. **Answer: B.** (Plain `EXISTS`, option A, finds the *opposite* — customers who *did* transact.)
-

Concept F: UNION vs UNION ALL (Q7)

- **Q7:** `UNION` deduplicates (extra sort/hash cost). `UNION ALL` keeps all rows including duplicates, faster. **Answer: B.**
-

Concept G: Window functions for running totals (Q8)

- **Q8:** `SUM(amount) OVER (PARTITION BY account_id ORDER BY txn_date)` computes a cumulative running total per account as rows are ordered by date. Plain `GROUP BY` collapses to one total per group, not a per-row running value. **Answer: A.**
-

Concept H: Composite indexes (Q5)

- **Q5:** A composite index on `(account_id, txn_date)` efficiently supports queries that filter on the leading column (`account_id =`) and range/sort on the trailing column (`txn_date`) in a **single index scan** — better than two separate single-column indexes, which would need an index intersection. **Answer: B.**

Brush up: know the general rule — composite index column order matters; leading column should be the equality filter, trailing column the range/sort.

Concept I: Sargability (Q15) — important, subtle

- **Q15:** Wrapping an indexed column in a function (`YEAR(txn_date)`, `MONTH(txn_date)`) makes the predicate **non-sargable** — the optimizer generally can't use the index, forcing a full scan. Instead use a direct range: `txn_date >= '2026-06-01' AND txn_date < '2026-07-01'`. **Answer: B.**

Brush up: "sargable" (Search ARGument ABLE) = a predicate that can use an index directly. Any function wrapped around a column in `WHERE` typically breaks sargability. This is a favorite "why is this query slow" interview topic.

Concept J: DELETE vs TRUNCATE vs DROP (Q16)

- **Q16: Answer: B.** `DELETE` = row-level, filterable, logged, rollback-able. `TRUNCATE` = removes all rows fast, minimal logging, resets identity, hard to roll back. `DROP` = removes the entire table structure.
-

Concept K: Self-joins for duplicate detection (Q17)

- **Q17:** Self-join on `(account_id, amount)` with `t1.txn_id <> t2.txn_id` finds transactions sharing account+amount with at least one other transaction — a duplicate-detection pattern. **Answer: B.**
-

Concept L: GROUP BY strictness (Q18)

- **Q18:** In strict SQL modes, every non-aggregated `SELECT` column must appear in `GROUP BY`. Here `type` is selected but neither aggregated nor grouped → invalid/undefined in strict engines. **Answer: B.**
-

Concept M: CTEs (Q19)

- **Q19:** CTEs (`WITH x AS (...)`) let you name and reuse intermediate result sets — mainly a **readability** win for multi-step logic, not an inherent performance guarantee. **Answer: B.**
-

Concept N: Isolation levels — dirty reads (Q20) — fintech-specific, important

- **Q20:** `READ UNCOMMITTED` allows **dirty reads** — reading another transaction's uncommitted changes, which might later roll back, leading to incorrect balance calculations. Fintech systems typically need at least `READ COMMITTED`, often `REPEATABLE READ`/`SERIALIZABLE` for money-movement. **Answer: B.**
-

Brush up: know the 4 isolation levels (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`) and which anomaly each prevents (dirty read, non-repeatable read, phantom read).

Priority Concept Checklist (rank order for tonight)

1. **RANK/DENSE_RANK/ROW_NUMBER tie behavior** — appears 3x, very testable
2. **NOT IN + NULL trap / three-valued logic** — classic silent-bug gotcha
3. **finally always runs**, catch-order compile errors
4. **Integer cache (-128 to 127) and String pool vs `new String()`**
5. **Static method hiding vs instance method overriding**; constructor-calls-overridden-method trap

6. **Composite index column order + sargability** (function-wrapped WHERE clauses)
 7. **WHERE vs HAVING, NOT EXISTS vs EXISTS**
 8. **ConcurrentHashMap vs Hashtable vs synchronizedMap;**
HashMap/LinkedHashMap/TreeMap ordering+complexity
 9. **BigDecimal vs double** for money; construct from String not double
 10. **Isolation levels** and dirty reads
 11. Pre/post increment tracing, ternary autoboxing promotion (lower priority — rare in practice, but shows up as a "gotcha" question)
 12. Stream laziness (intermediate vs terminal ops)
-

LeetCode Problems to Drill Tonight (by concept, not difficulty-max — pick 1-2 per bucket given your time)

Given the OA leans **conceptual/output-prediction** rather than heavy algorithmic DSA, I'd split your remaining prep time roughly 70% re-reading the concept table above / re-deriving each "why" out loud, 30% light DSA warm-up so your coding muscle is warm if there's a coding section too. Suggested light set:

- **Two Sum** (#1) — sanity check on HashMap basics in Java syntax
- **Group Anagrams** (#49) — HashMap<String, List<>> usage, tests collection fluency
- **Valid Parentheses** (#20) — Stack/Deque syntax in Java
- **Merge Intervals** (#56) — sorting + comparator syntax (relevant since TreeMap/sorting came up in SQL context too)
- **LRU Cache** (#146) — if you have 30+ min spare; directly touches LinkedHashMap (which was Q14) and is a very common fintech-adjacent OA question
- **Number of Islands** (#200) — if there's any graph/BFS-DFS component expected; low priority unless the OA description mentions it

Given this OA looks like it's almost entirely **language-fundamentals + SQL**, not classic algorithmic DSA, I'd actually deprioritize grinding new LeetCode tonight and instead spend the bulk of your time re-deriving *why* each answer above is correct out loud, in your own words, without looking — that's the fastest way to make sure you can handle a *variant* of these questions (e.g., a different Integer range, a different isolation level scenario) rather than just recognizing the exact same question.

Good luck tomorrow.